



EE5114: Autonomous Robot Navigation

Continuous Assessment 1

Name: Lu Zixuan

Student ID: A0285204J

29/September/2024

1 Introduction

The aim of this project is to implement an Extended Kalman Filter (EKF) to estimate the position and velocity of a UAV (Unmanned Aerial Vehicle). The primary challenge involves converting GPS coordinates (longitude, latitude and altitude) into NED (North, East, Down) coordinates, followed by utilizing the EKF to enhance the accuracy of the UAV's trajectory.

The given dataset includes the UAV's flight patterns and the initial step is to analyze the original plots to identify key flight events such as takeoffs and landings. Subsequently, the positional information in Geodetic coordinates, specifically longitude, latitude, and altitude, will be transformed into x, y, z positions within the NED coordinate system. Following this, the EKF will be implemented to handle sensor fusion, which combines the high-frequency IMU data with the lower-frequency GPS measurements. The inclusion of accelerometer bias in the EKF implementation aims to ensure that the velocity and position estimations are corrected over time.

This report will detail the steps taken to convert the GPS data, implement the EKF and evaluate its performance through various plots and figures. The analysis will focus on the UAV's flight behavior, estimation of state variables and the handling of sensor biases, particularly the accelerometer bias.

2 Questions and Solutions

2.1 Question 1

By understanding the original codes and observing the default plots of this set of data, can you deduce how many times the UAV had taken off and landed?

By analyzing the provided plots, including the Accelerations, Euler Angles and Motor Signals, it is evident that the UAV performed two takeoff and landing cycles.

The graphs of Acceleration and Euler Angles show two distinct periods of significant acceleration activity, corresponding to takeoffs and landings. Additionally, the Motor Signals plot exhibits two separate intervals where the motor inputs increase and stabilize during flight, followed by a decrease, signifying two complete flight events.

Thus, based on these observations, it can be deduced that the UAV took off and landed twice during the data collection period.

2.2 Question 2

When did the UAV take off and land for the 2nd time? Please provide relevant figures to support your explanations.

As shown in Figure 1, the UAV's altitude remains constant at around 1398 meters until the point marked at 1446.67 seconds, where the altitude starts to rise. This indicates that the UAV has taken off. The altitude rises and stabilizes slightly above 1400 meters, and eventually drops back down to 1396.86 meters at 1687.76 seconds, indicating the UAV has landed.

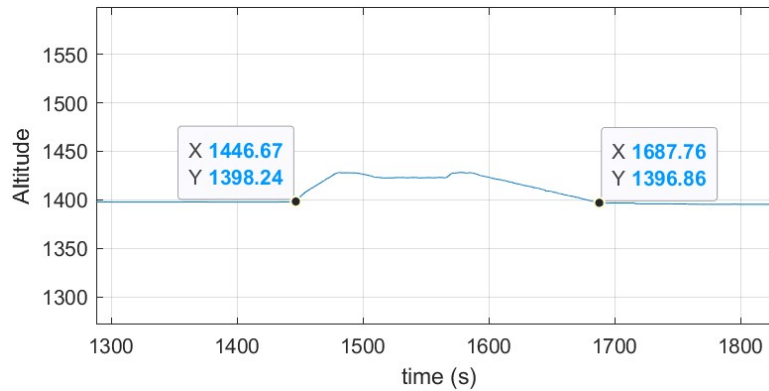


Figure 1: Altitude versus time

As shown in Figure 2, the motor input maintains a value of 1000 from 1379.6s until a significant rise occurs at 1439.76s, indicating the UAV's takeoff. The motor signals stabilize and remain elevated during the flight, eventually dropping back down to 1000 at 1695.7 seconds, indicating the motors were powered down for landing. So the UAV's second takeoff and landing times are 1439.76s and 1695.7s respectively according to the Motor signal.

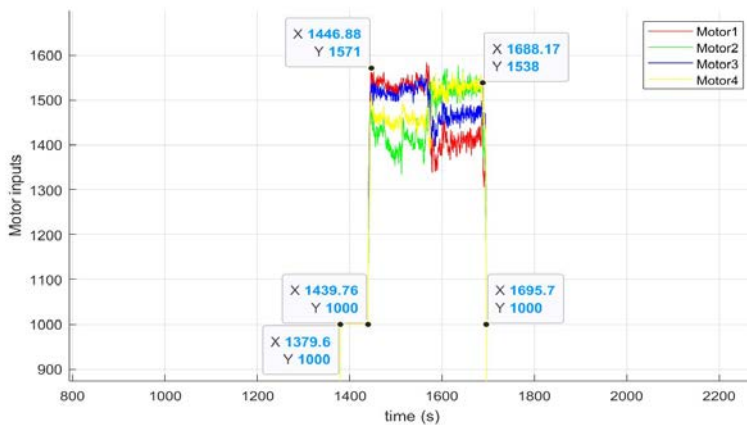


Figure 2: Motor signals versus time

- Motor signals provide a more direct indication: Motor signals reflect changes in the UAV's operational state more directly and rapidly. During takeoff, the motor input increases significantly, while after landing, the motor power decreases or returns to idle. Therefore, sharp changes in motor signals offer precise indicators of takes off and lands.
- Potential lag in altitude data: Although altitude data can indicate the UAV's flight status, changes in altitude usually occur after the motors have already been activated, which may introduce a few seconds of delay. As shown in Figure 1, when the motor input reaches its maximum value for the first time, at approximately 1446.88 seconds, the UAV's altitude has only just begun to change.
- Therefore, the final determined times for the UAV's second takeoff and landing are 1439.76 s and 1695.7 s.

2.3 Question 3

List the formulas you have used to convert GPS coordinates to NED positions and attach your corresponding codes for this step.

Identifying Valid GPS Data

To ensure the reliability of GPS data, it is crucial to identify when the signal becomes valid based on two key parameters: the number of visible satellites (`gps_nSat`) and the GPS fix status (`fix`). A valid GPS signal requires a minimum of four satellites (`gps_nSat` ≥ 4), allowing accurate 3D positioning, and a fix status of 3, indicating a successful 3D fix.

The dataset is evaluated row by row, checking for the first instance where both conditions are met. The earliest row satisfying `gps_nSat` ≥ 4 and the value of `fix` = 3 is defined as the starting point for reliable GPS data. This ensures that only valid and accurate data is used for subsequent analysis.

Code-description: Identifying Valid GPS Signals

```
% initialize the start_row
start_row = -1;
% Find the row where both gps_nSat >= 4 and fix == 3
for i = 1:length(gps_nSat)
    if gps_nSat(i) >= 4 && fix(i) == 3
        start_row = i;
        break; % Exit after finding the first matching row
    end
end

% If no matching row is found
if start_row == -1
    disp('GPS positioning ability and accuracy are low');
end
```

GPS to NED Transformation

Step 1: Conversion from Geodetic to ECEF Coordinates

The geodetic coordinates (ϕ, λ, h) , where ϕ is the latitude, λ is the longitude and h is the height above the ellipsoid, are converted into ECEF coordinates (X_e, Y_e, Z_e) using the following formulas:

$$X_e = (N(\phi) + h) \cos \phi \cos \lambda \quad (1)$$

$$Y_e = (N(\phi) + h) \cos \phi \sin \lambda \quad (2)$$

$$Z_e = \left(\frac{b^2}{a^2} N(\phi) + h \right) \sin \phi \quad (3)$$

Where $N(\phi)$ is the prime vertical radius of curvature, given by:

$$N(\phi) = \frac{a^2}{\sqrt{a^2 \cos^2 \phi + b^2 \sin^2 \phi}} = \frac{a}{\sqrt{1 - e^2 \sin^2 \phi}} \quad (4)$$

- a : Equatorial radius (semi-major axis)
- b : Polar radius (semi-minor axis)
- $e^2 = 1 - \frac{b^2}{a^2}$: Square of the first numerical eccentricity of the ellipsoid
- $N(\phi)$: Prime vertical radius of curvature

Step 2: Conversion from ECEF to Local NED Coordinates

The local NED coordinate system is defined with the origin at a reference point (X_{e0}, Y_{e0}, Z_{e0}) in ECEF coordinates. The transformation from ECEF to NED coordinates involves a translation followed by a rotation. First, translate the ECEF coordinates by subtracting the reference point:

$$\begin{bmatrix} \Delta X_e \\ \Delta Y_e \\ \Delta Z_e \end{bmatrix} = \begin{bmatrix} X_e - X_{e0} \\ Y_e - Y_{e0} \\ Z_e - Z_{e0} \end{bmatrix} \quad (5)$$

Next, apply a rotation matrix R^T to convert the translated ECEF coordinates into NED coordinates (X_n, Y_n, Z_n) :

$$\begin{bmatrix} X_n \\ Y_n \\ Z_n \end{bmatrix} = R^T \begin{bmatrix} \Delta X_e \\ \Delta Y_e \\ \Delta Z_e \end{bmatrix} \quad (6)$$

Where the rotation matrix R is given by:

$$R = \begin{bmatrix} -\sin \phi_0 \cos \lambda_0 & -\sin \lambda_0 & -\cos \phi_0 \cos \lambda_0 \\ -\sin \phi_0 \sin \lambda_0 & \cos \lambda_0 & -\cos \phi_0 \sin \lambda_0 \\ \cos \phi_0 & 0 & -\sin \phi_0 \end{bmatrix} \quad (7)$$

- ϕ_0 : latitude of the reference point in the geodetic coordinate
- λ_0 : longitude of the reference point in the geodetic coordinate

Final NED Coordinates

$$\begin{bmatrix} X_n \\ Y_n \\ Z_n \end{bmatrix} = R^T \begin{bmatrix} X_e - X_{e0} \\ Y_e - Y_{e0} \\ Z_e - Z_{e0} \end{bmatrix} \quad (8)$$

Where: X_n, Y_n, Z_n represents the north direction, the east direction and the down direction in the local coordinate system.

```

Code-description: Convert GPS raw measurements to local NED position
% Define reference point (initial latitude, longitude, altitude)
lat0 = deg2rad(lat(1)); % Convert reference latitude to radians
lon0 = deg2rad(lon(1)); % Convert reference longitude to radians
alt0 = alt(1); % Reference altitude in meters

% Earth's ellipsoid parameters for WGS-84
a = 6378137; % Earth's equatorial radius (semi-major axis) in meters
b = 6356752; % Earth's polar radius (semi-minor axis) in meters
e2 = 1 - (b^2 / a^2); % First eccentricity squared

% Convert the current latitude, longitude to radians
lat_rad = deg2rad(lat);
lon_rad = deg2rad(lon);
% Prime vertical radius of curvature at reference point
N0 = a / sqrt(1 - e2 * sin(lat0)^2);

% Convert reference point (lat0, lon0, alt0) to ECEF
X_e0 = (N0 + alt0) * cos(lat0) * cos(lon0);
Y_e0 = (N0 + alt0) * cos(lat0) * sin(lon0);
Z_e0 = (b^2 / a^2 * N0 + alt0) * sin(lat0);

% Initialize NED coordinate arrays
x_ned = zeros(size(lat)); % North (m)
y_ned = zeros(size(lat)); % East (m)
z_ned = zeros(size(lat)); % Down (m)

% Loop through each GPS point and convert it to NED
for i = 1:length(lat)
    % Compute prime vertical radius of curvature at current latitude
    N = a / sqrt(1 - e2 * sin(lat_rad(i))^2);

    % Convert current GPS (lat, lon, alt) to ECEF coordinates
    X_e = (N + alt(i)) * cos(lat_rad(i)) * cos(lon_rad(i));
    Y_e = (N + alt(i)) * cos(lat_rad(i)) * sin(lon_rad(i));
    Z_e = (b^2 / a^2 * N + alt(i)) * sin(lat_rad(i));

    % Compute the difference in ECEF coordinates
    delta_X = X_e - X_e0;
    delta_Y = Y_e - Y_e0;
    delta_Z = Z_e - Z_e0;

    % Apply rotation matrix to convert from ECEF to NED
    % Rotation matrix based on reference point's latitude and longitude
    R = [-sin(lat0) * cos(lon0), -sin(lat0) * sin(lon0), cos(lat0);
         -sin(lon0), cos(lon0), 0;
         -cos(lat0) * cos(lon0), -cos(lat0) * sin(lon0), -sin(lat0)];

```

```

Code-description: Final NED Coordinate
% Apply rotation to get NED coordinates
ned_coords = R * [delta_X; delta_Y; delta_Z];

x_ned(i) = ned_coords(1); % North
y_ned(i) = ned_coords(2); % East
z_ned(i) = ned_coords(3); % Down
end

```

The Earth's equatorial radius (semi-major axis) is $a=6,378,137$ meters and the Earth's polar radius (semi-minor axis) is $b=6,356,752$ meters based on the World Geodetic System 1984 (WGS84) model. The final converting results are as follows:

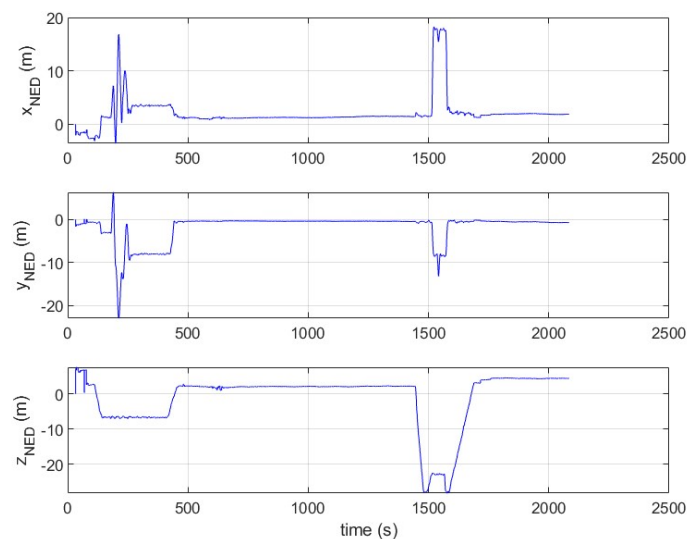


Figure 3: GPS to Local NED

2.4 Question 4

List the formulas you have used for the implementation of Kalman filter and attach your corresponding codes for this step.

Extended Kalman Filter (EKF) Operational Steps

The Extended Kalman Filter (EKF) is an extension of the standard Kalman filter, used for nonlinear systems where either the state transition model or the measurement model is nonlinear. EKF linearizes the state and measurement equations using the Jacobian matrices at each time step.

Step 1. Prediction Step

In the prediction step, the EKF uses the previous state estimate and the state transition function to predict the current state and covariance matrix.

The state prediction equation is:

$$\hat{x}_{k|k-1} = f(\hat{x}_{k-1|k-1}, u_{k-1}) \quad (9)$$

where f is the nonlinear state transition function, and $\hat{x}_{k|k-1}$ is the predicted state at time k , based on the state estimate from time $k - 1$.

The covariance prediction equation is:

$$P_{k|k-1} = F_{k-1} P_{k-1|k-1} F_{k-1}^T + Q \quad (10)$$

where $P_{k|k-1}$ is the predicted covariance matrix, $P_{k-1|k-1}$ is the covariance from the previous step, and Q is the process noise covariance matrix. F_{k-1} is the Jacobian matrix.

Step 2. Correction Step

The correction step combines the actual measurements with the predicted state to refine the state estimate and update the covariance matrix.

The Kalman Gain K_k is computed as follows:

$$K_k = P_{k|k-1} H_k^T (H_k P_{k|k-1} H_k^T + R)^{-1} \quad (11)$$

where H_k is the Jacobian matrix of the measurement model, and R is the measurement noise covariance matrix. The Kalman gain balances the uncertainty between the predicted state and the measurements.

The state update equation is:

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k (y_k - H_k \hat{x}_{k|k-1}) \quad (12)$$

where y_k is the actual measurement at time k , and $H_k \hat{x}_{k|k-1}$ is the predicted measurement based on the predicted state.

Through the state update, the predicted state is corrected using the Kalman gain and the difference between the actual measurement and the predicted measurement.

The covariance update equation is:

$$P_{k|k} = (I - K_k H_k) P_{k|k-1} \quad (13)$$

This updates the covariance matrix to reflect the reduced uncertainty after incorporating the measurements.

```

Code-description: Implement EKF to estimate NED position and velocity
% Precompute the rotation matrix function
R_gb = @(phi, tht, psi) [cos(tht)*cos(psi), cos(tht)*sin(psi), -sin(tht);
                        sin(phi)*sin(tht)*cos(psi)-cos(phi)*sin(psi),
                        ↪ sin(phi)*sin(tht)*sin(psi)+cos(phi)*cos(psi),
                        ↪ sin(phi)*cos(tht);
                        cos(phi)*sin(tht)*cos(psi)+sin(phi)*sin(psi),
                        ↪ cos(phi)*sin(tht)*sin(psi)-sin(phi)*cos(psi),
                        ↪ cos(phi)*cos(tht)];

% EKF Implementation
for k = 2:length(t)
    dt = t(k) - t(k-1);

    % Get the current rotation matrix from body to global frame
    R_g_b = R_gb(phi(k), tht(k), psi(k));

    % 1. Prediction Step
    % F matrix with rotation matrix R_g_b incorporated
    F = [1, 0, 0, dt, 0, 0, -dt^2/2, 0, 0;
        0, 1, 0, 0, dt, 0, 0, -dt^2/2, 0;
        0, 0, 1, 0, 0, dt, 0, 0, -dt^2/2;
        0, 0, 0, 1, 0, 0, -dt, 0, 0;
        0, 0, 0, 0, 1, 0, 0, -dt, 0;
        0, 0, 0, 0, 0, 1, 0, 0, -dt;
        0, 0, 0, 0, 0, 0, 1, 0, 0;
        0, 0, 0, 0, 0, 0, 0, 1, 0;
        0, 0, 0, 0, 0, 0, 0, 0, 1];

    G = [dt^2/2, 0, 0;
        0, dt^2/2, 0;
        0, 0, dt^2/2;
        dt, 0, 0;
        0, dt, 0;
        0, 0, dt;
        0, 0, 0;
        0, 0, 0;
        0, 0, 0];

    % Compute the control input: accelerometer readings in global frame
    g = -9.81; % Gravity constant
    u_k = R_g_b * [acx(k); acy(k); acz(k)-g];
    % Subtract gravity in the z-axis

    % State prediction
    x_pred = F * x_est(:, k-1) + G * u_k;

    % Covariance prediction
    P_pred = F * P * F' + Q;

```

Code-description: EKF Implementation

```
% 2. Correction Step
H_k = [eye(3), zeros(3, 6)]; % Measurement matrix

% Modify the measurement vector z_k to only include position data
z_k = [x_ned(k); y_ned(k); z_ned(k)];

% Kalman gain
K = P_pred * H_k' / (H_k * P_pred * H_k' + R);

% Update state estimate
x_est(:, k) = x_pred + K * (z_k - H_k * x_pred);

% Update the covariance matrix
P = (eye(9) - K * H_k) * P_pred;
end
```

The results of EKF implementation of NED are as follows:

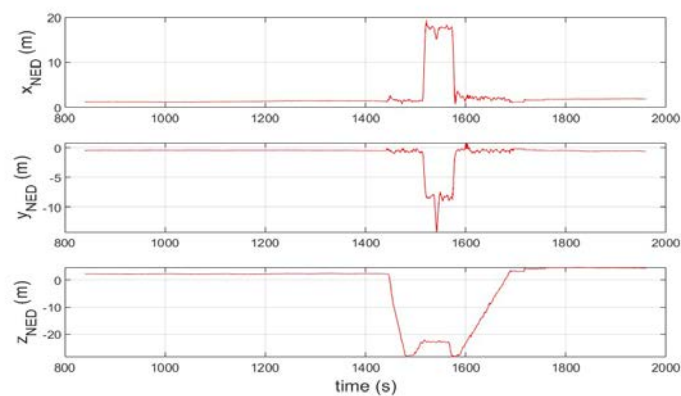


Figure 4: EKF estimated NED position

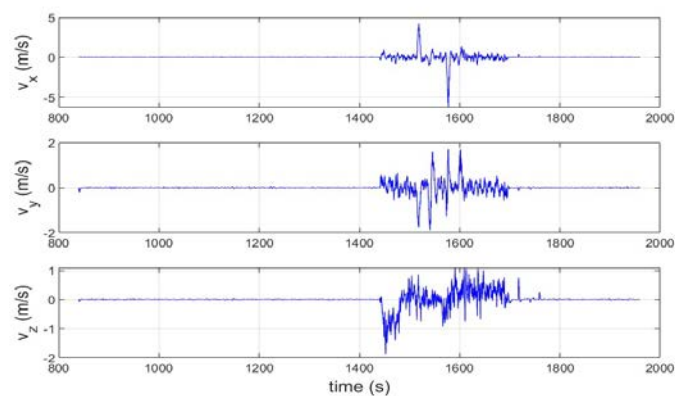


Figure 5: EKF estimated NED velocity

2.5 Question 5

Please explain the physical meanings of your state variables used in your EKF and which coordinate system they are defined in.

- State Variables:
 - $x_{ned}, y_{ned}, z_{ned}$: Position in the NED coordinate system.
 - $v_{x_{ned}}, v_{y_{ned}}, v_{z_{ned}}$: Velocity in the NED coordinate system.
 - $bias_{ax}, bias_{ay}, bias_{az}$: Accelerometer bias in the x, y, and z directions.
- Coordinate System:

The entire EKF process works in the NED coordinate system.

Code-description: State vector define

```
% State vector:
[x_ned, y_ned, z_ned, vx_ned, vy_ned, vz_ned, bias_ax, bias_ay, bias_az]
x_est = zeros(9, length(t)); % Estimated state over time
x_est(:,1) = [x_ned(1); y_ned(1); z_ned(1); 0; 0; 0; 0; 0; 0];
% Initialize state (position from GPS, velocity and bias set to zero)
```

2.6 Question 6

Note that GPS update rate is slower than IMU update rate for this set of data. Please explain how you have implemented your code to address this practical issue.

Since the GPS data updates at a lower frequency compared to the IMU, the EKF prediction step is executed at every IMU update, ensuring continuous estimation of the system state based on the high-frequency IMU data. The correction step, which incorporates the GPS measurements, is only executed when new GPS data becomes available. This is determined by checking whether the current GPS position data (latitude, longitude, altitude) has changed compared to the previous time step.

In detail, a variable `gps_available(k)` was introduced to flag whether GPS data is available at each time step. If GPS data at the current step differs from the previous step, the variable is set to true, allowing the EKF to use the GPS measurements in the correction step. When no new GPS data is available (`gps_available(k) = false`), the EKF only performs the prediction step, which is driven by IMU measurements, ensuring that the system state continues to be updated without relying on less frequent GPS updates.

By decoupling the prediction and correction steps in this way, the filter can handle the higher IMU update rate while still benefiting from the GPS data when it becomes available.

```

Code-description: Processing of IMU and GPS update
% Initialize the gps_available array
gps_available = false(length(t), 1);

% Check GPS data availability
for k = 2:length(t)
    % If the current GPS data differs from the previous time step,
    ↪ consider GPS data updated
    gps_available(k) = (lat(k) ~= lat(k-1)) || (lon(k) ~= lon(k-1)) ||
    ↪ (alt(k) ~= alt(k-1));
end
gps_available(1) = true;

%%EKF Implementation(in Q4)
for k = 2:length(t)
    ...

    % State prediction
    x_pred = F * x_est(:, k-1) + G * u_k;

    % Covariance prediction
    P_pred = F * P * F' + Q;

    % Check if GPS data is available
    if gps_available(k)
        % 2. Correction Step
        H_k = [eye(3), zeros(3, 6)];

        % Modify the measurement vector z_k
        z_k = [x_ned(k); y_ned(k); z_ned(k)];

        % Kalman gain
        K = P_pred * H_k' / (H_k * P_pred * H_k' + R);

        % Update state estimate
        x_est(:, k) = x_pred + K * (z_k - H_k * x_pred);

        % Update the covariance matrix
        P = (eye(9) - K * H_k) * P_pred;
    else
        % If no GPS data is available, only perform prediction, no
        ↪ correction
        x_est(:, k) = x_pred;
        P = P_pred;
    end
end
end

```

2.7 Question 7

Please explain what values you have chosen to initialize the state variables and the state covariance matrix P. Why these values?

State variables

- **Position:** The initial position values are set using the GPS measurement, $x_{ned}(1)$, $y_{ned}(1)$, and $z_{ned}(1)$. Before the second flight begins, the UAV remains in a relatively stable state. Therefore, the converted NED based on GPS data can be used as the initial position.
- **Velocity:** The initial velocity values are set to 0. This assumption is based on the fact that at the initial time, the system is considered stationary.
- **Biases:** The accelerometer biases (a_x , a_y , and a_z) are also initialized to 0. Before the flight begins, the UAV is stationary; thus, the accelerometer bias is set to zero at this time.

state covariance matrix P

- **Position uncertainty:** Since the initial GPS position is relatively reliable, the corresponding diagonal elements of the covariance matrix for position ($P_{x,y,z}$) are set to smaller values, such as 0.01.
- **Velocity and bias uncertainty:** As the initial velocity and bias estimates are less certain (since they are set to zero by assumption), the corresponding diagonal elements of the covariance matrix for velocity and bias ($P_{vx,vy,vz, P_{bx,by,bz}}$) are initialized with larger values, which is 0.1 and 1. This allows the filter to adapt and update these values more significantly in the early stages of estimation.

```
P = diag([0.01, 0.01, 0.01, 0.1, 0.1, 0.1, 1, 1, 1])
% Initial state covariance matrix
```

2.8 Question 8

Please explain your choice of Q and R matrices for your EKF implementation.

Process Noise Covariance Matrix Q:

Q represents the noise in the control process and reflects the accuracy of the system model. Due to the UAV taking off and landing during the process, the entire system exhibits significant dynamic changes; therefore, values related to velocity and acceleration are relatively large. Especially when the UAV ascends and descends rapidly, the process noise can be appropriately increased, particularly for the elements related to the acceleration bias.

By keeping other parameters constant and adjusting values of one single type to compare the resulting images, a series of tests were conducted. Ultimately, the value related to position

is set to 0.1, the value related to velocity is set to 0.2, and the value related to acceleration is set to 0.5.

```
Q = diag([0.1, 0.1, 0.1, 0.2, 0.2, 0.2, 0.5, 0.5, 0.5]);
% Process Noise Covariance Matrix
```

Measurement Noise Covariance Matrix R :

R reflects the reliability of the measurements. The smaller this value, the more the results rely on the measured data. To account for potential impacts on the GPS and IMU during flight, this value was set higher. Based on a series of tests, the final value was determined to be 0.8 for this parameter.

```
R = diag([0.8, 0.8, 0.8]);
% Measurement Noise Covariance Matrix
```

2.9 Question 9

Have the accelerometer bias values converged in the end? What values did they converge on?

The accelerometer bias gradually converged to stable values, with significant fluctuations occurring during the second takeoff and landing phase.

- The bias in the a_x axis converged to around -0.002 m/s^2 .
- The bias in the a_y axis converged to around 0.24 m/s^2 .
- The bias in the a_z axis converged to around -0.06 m/s^2 .

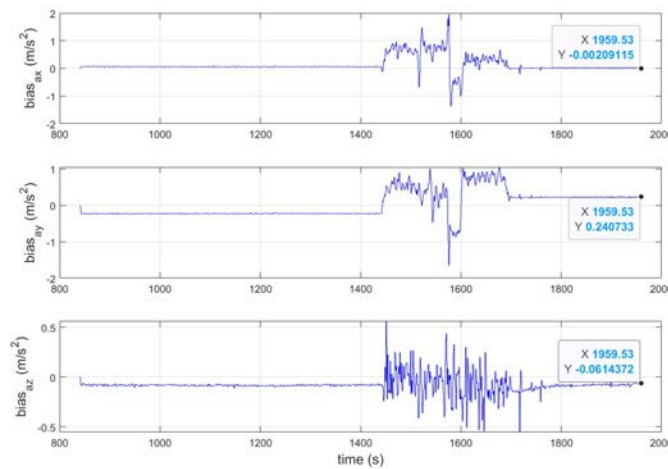


Figure 6: EKF Estimated Accelerometer Bias

2.10 Question 10

If considering accelerometer bias is not a constant, but gradually drifts over time, how will you modify the implementation of EKF to address this issue?

- If the accelerometer bias drifts over time, it can be modeled as a dynamic state variable, and a random walk model can be introduced to capture its changes.
- The process noise matrix Q has been modified, adjusting the values related to acceleration.
- In the prediction step, the control input should subtract the accelerometer bias. The correction is made by subtracting the bias from the accelerometer readings.

Code-description: Adjustment for bias drifting

```
% The model was updated to include the configuration for bias.
F = [1, 0, 0, dt, 0, 0, -dt^2/2, 0, 0;
     0, 1, 0, 0, dt, 0, 0, -dt^2/2, 0;
     0, 0, 1, 0, 0, dt, 0, 0, -dt^2/2;
     0, 0, 0, 1, 0, 0, -dt, 0, 0;
     0, 0, 0, 0, 1, 0, 0, -dt, 0;
     0, 0, 0, 0, 0, 1, 0, 0, -dt;
     0, 0, 0, 0, 0, 0, 1, 0, 0;
     0, 0, 0, 0, 0, 0, 0, 1, 0;
     0, 0, 0, 0, 0, 0, 0, 0, 1];

...
Q = diag([0.1, 0.1, 0.1, 0.2, 0.2, 0.2, 1, 1, 1]);
% The value related to bias noise was increased accordingly.
...
u_k = R_g_b * [acx(k) - x_est(7, k-1); acy(k) - x_est(8, k-1); acz(k) -
    ↪ g - x_est(9, k-1)];
% In the prediction step, the control input should account for the
    ↪ accelerometer bias by subtracting it.
```